FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate

Bài 5: Xử Lý Dữ Liệu Chuỗi Thời Gian

www.poly.edu.vn

AGENDA

- Dữ liệu chuỗi thời gian
- Vấn đề xu hướng
- Vấn đề mùa vụ
- Thực hành

1. Dữ Liệu Chuỗi Thời Gian

1.1 Đặc điểm

- Dữ liệu được ghi nhận theo trình tự thời gian t1, t2, ..., tn.
- Mỗi giá trị được gắn với thời điểm cụ thể.
- Thứ tự dữ liệu quan trọng – không được sắp xếp ngẫu nhiên.
- Dữ liệu có thể hằng ngày, hằng tuần, hằng tháng, hằng quý.

1.2 Ví dụ dữ liệu

Ví dụ 1: Giá nhà theo tháng

Tháng	Giá bán (triệu VND/m²)
01/2025	20.0
02/2025	20.2
03/2025	20.1
04/2025	20.5

1.2 Ví dụ dữ liệu

Ví dụ 2: Lưu lượng khách truy cập web theo ngày

Ngày	Lượt truy cập
01-10-2025	1200
02-10-2025	1150
03-10-2025	1300
04-10-2025	1250

1.3 Ứng dụng

- Dự báo giá nhà (Housing Price Forecasting)

Mục tiêu: Dự đoán giá bán trong tương lai dựa trên dữ liệu lịch sử.
Cách dùng

- Nhận diện xu hướng dài hạn: giá nhà tăng hay giảm trung bình mỗi năm.
- Phát hiện mùa vụ: mùa bán nhà cao điểm (ví dụ hè).
- Mô hình dự báo: ARIMA, Prophet, LSTM.

Lợi ích:

- Giúp người mua/bán ra quyết định đúng thời điểm.
- Hỗ trợ ngân hàng, quỹ đầu tư định giá bất động sản.

1.3 Ứng dụng

- Dự báo nhu cầu thị trường (Demand Forecasting)

Mục tiêu: Ước lượng nhu cầu sản phẩm, dịch vụ theo thời gian.
Cách dùng

- Dữ liệu bán hàng hàng ngày/tuần/tháng → nhận dạng xu hướng và mùa vụ.
- Phân tích peaks (ngày lễ, Tết) để chuẩn bị tồn kho.
- Mô hình: Holt-Winters, Prophet, SARIMA.

Lợi ích:

- Giảm tồn kho dư thừa, tối ưu chi phí logistics.
- Đảm bảo đủ hàng đáp ứng nhu cầu cao điểm.

1.3 Ứng dụng

- Phân tích tài chính (Financial Analysis & Forecasting)

Mục tiêu: Dự báo giá cổ phiếu, tỷ giá, lợi nhuận.
Cách dùng

- Nhận diện trend thị trường: tăng/giảm dài hạn.
- Phát hiện biến động định kỳ: tuần, tháng, mùa kinh doanh.
- Dùng mô hình ARIMA, GARCH, LSTM để dự báo rủi ro.

Lợi ích:

- Hỗ trợ quyết định đầu tư, mua bán cổ phiếu, quản lý rủi ro.
- Giúp lập kế hoạch tài chính chiến lược.

1.3 Ứng dụng

- Quản lý chuỗi cung ứng (Supply Chain Management)

Mục tiêu: Dự đoán nhu cầu, tối ưu kho bãi và vận chuyển.
Cách dùng

- Dữ liệu tồn kho + bán hàng → phân tích xu hướng và seasonality.
- Dự báo nhu cầu từng cửa hàng, từng khu vực.
- Kết hợp mô hình dự báo + tối ưu hóa logistics.

Lợi ích:

- Giảm chi phí vận chuyển, tồn kho.
- Ngăn ngừa thiếu hàng trong mùa cao điểm.

1.4 Vấn đề thường gặp

1.4.1 Không đồng nhất định dạng

Ngày	Giá nhà (triệu VNĐ)
2025-01-05	20.0
05/02/2025	20.3
Mar 05, 2025	20.5
2025/04/05	21.0

1.4 Vấn đề thường gặp

1.4.2 Giá trị trống (Missing Values)

Ví dụ: Tháng 2 giá nhà bị thiếu → không thể tính trung bình 3 tháng chính xác.

Ngày	Giá nhà (triệu VNĐ)
2025-01-05	20.0
2025-02-05	NaN
2025-03-05	20.5
2025-04-05	21.0

1.4 Vấn đề thường gặp

1.4.3 Dữ liệu không đều / Tần suất thay đổi

Ngày	Giá nhà (triệu VNĐ)
2025-01-05	20.0
2025-01-10	20.2
2025-01-11	20.3
2025-01-20	20.5

- Khoảng cách giữa các ngày không đều (5 ngày → 1 ngày → 9 ngày).
- Nếu muốn phân tích theo ngày, cần resampling để dữ liệu đồng nhất.

2. Phương Pháp Xử Lý

2.1 Chuyển đổi định dạng thời gian

Vấn đề: Trong cùng một tập dữ liệu, cột thời gian có thể xuất hiện nhiều định dạng khác nhau:

- "2025-01-05" (YYYY-MM-DD)
- "05/01/2025" (DD/MM/YYYY)
- "Jan 5, 2025" (MMM D, YYYY)
- "2025/01/05 14:30" (YYYY/MM/DD HH:MM)

Khi đọc dữ liệu vào Python/pandas: Chưa chuẩn hóa → các giá trị được coi là string → không thể sắp xếp, trích xuất đặc trưng, hay resampling.

2.1 Chuyển đổi định dạng thời gian

Giải pháp: Dùng pd.to_datetime() với tham số

- format='mixed' -> Cho phép pandas tự động phát hiện và xử lý nhiều định dạng ngày tháng khác nhau trong cùng một cột.
- dayfirst=True -> Hiểu là ngày trước: DD/MM/YYYY. Tùy dữ liệu từng quốc gia, khu vực.
- errors='coerce' -> Khi có chuỗi ngày tháng không thể nhận dạng được, pandas sẽ: Gán giá trị NaT (Not a Time) thay vì báo lỗi.

2.1 Chuyển đổi định dạng thời gian

Ví dụ minh họa

```python
import pandas as pd

# Dữ liệu có nhiều định dạng thời gian khác nhau
data = {
    'date': ["2025-01-05", "05/01/2025", "Jan 5, 2025", "2025/01/06 14:30", "random text"],
    'price': [20.0, 20.1, 20.2, 20.3, 221]
}

df = pd.DataFrame(data)
print("Dữ liệu gốc:")
print(df)

# Chuyển sang datetime, cho phép nhiều định dạng khác nhau
df['date'] = pd.to_datetime(df['date'], format='mixed', dayfirst=True, errors='coerce')

print("\n Sau khi chuẩn hóa về datetime:")
print(df)

# Chuẩn hóa định dạng xuất (nếu cần)
df['date_str'] = df['date'].dt.strftime('%Y-%m-%d')
print("\n Dữ liệu sau khi chuẩn hóa thành chuỗi YYYY-MM-DD:")
print(df)
```

2.2 Chuẩn hóa tần suất

Resampling là quá trình chuyển đổi tần suất thời gian (time frequency) của dữ liệu chuỗi thời gian — tức là thay đổi khoảng cách giữa các điểm dữ liệu. Có thể thực hiện theo hai hướng:

- Downsampling - Giảm tần suất
- Upsampling - Tăng tần suất

2.2 Chuẩn hóa tần suất

2.2.1 Downsampling – Giảm tần suất

Ý nghĩa: Tổng hợp dữ liệu chi tiết (theo giờ, ngày) thành mức thưa hơn (tuần, tháng, quý).

Mục đích:

- Làm mượt dữ liệu, giảm nhiễu ngắn hạn.
- Dễ quan sát xu hướng dài hạn.

Ví dụ:

- Doanh thu theo ngày → tổng hợp theo tuần.
- Nhiệt độ theo giờ → trung bình theo ngày.

2.2 Chuẩn hóa tần suất

2.2.1 Downsampling – Giảm tần suất

Tình huống: Một cửa hàng có dữ liệu doanh thu theo ngày, nhưng bạn muốn xem doanh thu theo tuần để nhìn tổng thể xu hướng kinh doanh.

- Tuần đầu (1–5/1) có tổng doanh thu = 10+12+11+13+9 = 55.
- Tuần sau (6–7/1) tổng = 29.

Ngày	Doanh thu (triệu)
2025-01-01	10
2025-01-02	12
2025-01-03	11
2025-01-04	13
2025-01-05	9
2025-01-06	14
2025-01-07	15

2.2 Chuẩn hóa tần suất

2.2.1 Downsampling – Giảm tần suất

```
import pandas as pd

# Tạo dữ liệu mẫu theo ngày
data = {
    'date': pd.date_range('2025-01-01', periods=7, freq='D'),
    'revenue': [10, 12, 11, 13, 9, 14, 15]
}
df = pd.DataFrame(data)

# Downsampling: tổng hợp theo tuần
df_weekly = df.resample('W', on='date').sum()

print(df_weekly)
```

2.2 Chuẩn hóa tần suất

2.2.2 Upsampling – Tăng tần suất

Ý nghĩa: Tăng độ chi tiết của dữ liệu bằng cách chèn thêm các mốc thời gian mới giữa các điểm sẵn có.

Mục đích:

- Tạo dữ liệu có tần suất đều cho mô hình dự báo (ARIMA, Prophet…).
- Phục vụ trực quan hóa hoặc nội suy xu hướng ngắn hạn.

Ví dụ:

- Dữ liệu theo tháng → mở rộng thành từng ngày bằng cách điền giá trị (ffill, interpolate).
- Dữ liệu theo tuần → mở rộng thành ngày để vẽ biểu đồ mịn hơn.

2.2 Chuẩn hóa tần suất

2.2.2 Upsampling – Tăng tần suất

Tình huống: Dữ liệu doanh thu theo tháng, nhưng muốn phân tích theo ngày

```
# Upsampling: chuyển thành dữ liệu theo ngày
df_daily = df.resample('D', on='date').ffill()
```

`.resample('D')` -> tạo khung dữ liệu theo ngày.

`.ffill()` -> dùng giá trị gần nhất trước đó để điền vào các ngày trống.

Tháng	Doanh thu (triệu)
2025-01-01	310
2025-02-01	280
2025-03-01	350

2.2 Chuẩn hóa tần suất

2.2.2 Upsampling – Tăng tần suất

Các phương pháp điền dữ liệu

Phương pháp	Cách hoạt động	Phù hợp cho	Ghi chú
ffill() (Forward Fill)	Dùng giá trị trước đó để điền vào ô trống	Dữ liệu thay đổi chậm (doanh thu, giá trung bình)	Đơn giản, giữ ổn định
bfill() (Backward Fill)	Dùng giá trị sau đó để điền vào ô trống	Dữ liệu theo chu kỳ ngắn	Có thể làm mất tính thời gian
interpolate()	Tính giá trị trung gian giữa 2 điểm (nội suy)	Dữ liệu biến động liên tục (nhiệt độ, giá cổ phiếu)	Mượt và tự nhiên hơn

3. Nhận Dạng Đặc Trưng

3.1 Theo thời gian cơ bản

Đặc trưng	Cách tạo	Ý nghĩa / Ứng dụng
year	df['date'].dt.year	Phân tích xu hướng theo năm (trend dài hạn)
month	df['date'].dt.month	Phát hiện mùa vụ theo tháng (Tết, Noel, hè...)
day	df['date'].dt.day	Hữu ích khi dữ liệu chi tiết theo ngày
quarter	df['date'].dt.quarter	Hữu ích cho phân tích tài chính, báo cáo quý

Ví dụ: Doanh thu công ty tăng mạnh vào quý 4 hàng năm → đặc trưng quarter giúp mô hình nhận ra quy luật này.

3.2 Theo tuần

Đặc trưng	Cách tạo	Ý nghĩa
day_of_week	df['date'].dt.dayofweek *(Thứ Hai=0, Chủ Nhật=6)*	Hành vi thay đổi theo ngày trong tuần
is_weekend	(df['day_of_week'] >= 5).astype(int)	1 = Cuối tuần, 0 = Ngày thường
week_of_year	df['date'].dt.isocalendar().week	Theo dõi xu hướng hàng tuần

Ví dụ: Website có lưu lượng cao vào thứ 7, CN → is_weekend = 1 giúp mô hình nhận biết pattern.

3.3 Theo giờ/phút

Đặc trưng	Cách tạo	Ý nghĩa
hour	df['date'].dt.hour	Phân tích theo giờ trong ngày
minute	df['date'].dt.minute	Hữu ích với dữ liệu cảm biến, IoT
part_of_day *(sáng/chiều/tối)*	df['hour'].apply(...)	Nhận dạng hành vi theo khung giờ

Ví dụ: Lượt truy cập cao nhất vào 9h sáng và 20h tối → hour là đặc trưng rất mạnh khi dự báo nhu cầu.

3.3 Theo chu kỳ

Các biến như “tháng” hay “giờ” có tính chu kỳ — sau tháng 12 là tháng 1, sau 23h là 0h -> Mô hình tuyến tính có thể hiểu sai nếu không mã hóa đúng cách.

-> Mã hóa chu kỳ bằng hàm sin/cos để thể hiện “vòng lặp”.

```
df['month_sin'] = np.sin(2 * np.pi * df['month']/12)
df['month_cos'] = np.cos(2 * np.pi * df['month']/12)
```

4. Thực Hành

4.1 Nạp và chuẩn bị dữ liệu

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

plt.style.use("seaborn-v0_8")

df = pd.read_csv('data.csv')
df.head()
```

4.2 Chuẩn hóa thời gian

```
# Chuyển cột về datetime (dù đã đúng, vẫn demo)
df["date"] = pd.to_datetime(df["date"])

# Đặt index theo thời gian -> rất quan trọng cho chuỗi thời gian
df = df.set_index("date")

df.head()
```

4.3 Resampling

```
# Lấy trung bình giá theo từng tháng
df_monthly = df.resample("M").mean()

df_monthly.head()
```

4.4 Tạo đặc trưng thời gian

```
df_monthly["year"] = df_monthly.index.year
df_monthly["month"] = df_monthly.index.month
df_monthly["quarter"] = df_monthly.index.quarter

# Hàm xác định mùa vụ
def get_season(month):
    if month in [12,1,2]: return "winter"
    if month in [3,4,5]: return "spring"
    if month in [6,7,8]: return "summer"
    return "autumn"

df_monthly["season"] = df_monthly["month"].apply(get_season)
# One-hot encoding cho mùa vụ
df_monthly = pd.get_dummies(df_monthly, columns=["season"])

df_monthly.head()
```

4.5 Điền dữ liệu thiếu

```
# Forward Fill
df_ffill = df_monthly["price"].ffill()

# Backward Fill
df_bfill = df_monthly["price"].bfill()

df_missing_demo = pd.DataFrame({
    "original": df_monthly["price"],
    "ffill": df_ffill,
    "bfill": df_bfill
})
df_missing_demo.head(10)
```

4.6 Nội suy và làm mịn

```
# Nội suy
df_interp = df_monthly["price"].interpolate(method="linear")

# Làm mịn
df_monthly["MA_3"] = df_interp.rolling(window=3).mean()
df_monthly["MA_6"] = df_interp.rolling(window=6).mean()

plt.figure(figsize=(12,5))
plt.plot(df_interp, label="Interpolated", linewidth=1)
plt.plot(df_monthly["MA_3"], label="3-month MA", linewidth=2)
plt.plot(df_monthly["MA_6"], label="6-month MA", linewidth=2)
plt.title("Moving Average")
plt.legend()
plt.show()
```

4.7 Decomposition

```
# period=12 vì dữ liệu dạng tháng → chu kỳ 1 năm
result = seasonal_decompose(df_interp, model="additive", period=12)

result.plot()
plt.show()
```

FPT Education

FPT POLYTECHNIC

THANK YOU